Білет №1

1) Операції додавання (+) та віднімання (–) можуть бути як бінарними, так і унарними. Бінарні операції + та – використовуються у виразах при проведенні обчислень.
Бінарні оператори + попередньо визначені для числових і рядкових типів. Для числових типів оператор "+" обчислює суму двох його операндів. Якщо один або обидва операнда мають строковий тип, оператор "+" зв'язує рядкові уявлення операндів.
<оп.1><б.оп><оп.2> / <у.оп><оп.1>

2) Об'єкт в об'єктно-орієнтованому програмуванні - що має чітко виражене функціональне призначення в деякій області, належить до визначеного класу та характеризується своїми властивостями та поведінкою.
Клас – це якась безліч об'єктів, що мають загальну структуру й загальне поводження.

```
class ім’я класу
{
//оголошення змінних
доступ тип змінна 1;
доступ тип змінна 2;
//…
доступ тип змінна N;
// Оголошення методів
доступ тип_повернення метод 1 (параметри)
{
// тіло методу
}
}
```

Визначивши клас, ви можете розмістити в пам'яті будь-яке число відповідних об'єктів, використовуючи ключове слово C# new. При цьому, проте, слід розуміти, що ключове слово new повертає посилання на об'єкт у динамічній пам'яті, а не сам реальний об'єкт. Ця змінна з посиланням запам'ятовується в стеку для використання в додатку надалі. Для виклику членів об'єкта слід застосувати до збереженого посилання операцію C#, що позначається крапкою. Об’єкти мають тип посилання і створюються за допомогою ключового слова new: Name n = new Name() Тут змінна n не містить значення об’єкта Name, а містить адресу об’єкта (безіменного). Сама змінна n є лише посиланням на цей об’єкт.

Білет №2

1) оператори * / % (остача) бінарні оператори що здійснюються над операндами довільних цілих та
дійсних чисел
Зауваження.

- Операція % в мові C# може виконуватись і над операндами дійсних типів.
- ділення цілих чисел дає результат цілого типу
- Якщо один з операндів має тип з плаваючою комою, а другий операнд має цілий тип, то результатом має також тип з плаваючою комою

2) Конструктор – спеціальний метод що може бути визначеним і який викликається щоразу при створенні екзепляра класа, значення він не повертає. Задача конструктора полягає в тому, що він повинен створити об’єкт вказаного класу і перевести його в діючий стан.

```
class Person
{
int age;
string name;
}
```

Деструктор - спеціальний метод що виконує зворотні функції від конструктора. Призначений для звільнення памяті від об*єктів класу, також викликається коли об*єкт має бути видалений з памяті. ~<ім'я_класу>(){//тіло деструктора}

Білет №3

1)Унарний + та – це унарні оператори. Унарний + повертає значення операнда не змінюючи його. Операнд унарного + повинен мати арифметичний тип Console.WriteLine(+4); \\виведе 4
Унарний – повертає значення операнда змінивши його знак на протилежний.
Console.WriteLine(4); \\виведе -4
Console.WriteLine(-(-4)); \\виведе 4
2) Поле. Так називається член-змінна, яка має деяке значення. В ООП поля іноді називають даними об'єкта. До поля можна застосовувати кілька модифікаторів залежно від призначення цього поля. У число модифікаторів входять static, readonly і const. Public int a
Метод - Це реальний код, що впливає на дані об'єкта (або поля). Фактично, методи – це функції, які визначають певні дії з даними. Private string SetQuestion(string question){//тіло метода}

Білет №4

1) Операції інкременту і декременту використовують для збільшення та зменшення значення змінної на одиницю. Ці операції можуть вживатися у різних формі.
Постфіксна форма:
i++ i=i+1
i-- i=i-1
Префіксна форма:
++i i=i+1
--i i=i-1
Якщо унарну операцію вжито у префіксній формі у виразі, то вона виконується до використання значення змінної у виразі. Якщо ж унарну операцію вжито у постфіксній формі, то операція виконується після використання значення змінної у виразі.
2) Властивість – це два методи, призначені для роботи з закритим полем, і оформлені спеціальним чином. Як правило, властивість пов'язана із закритим полем класу й визначає методи його одержання й установки. Синтаксис властивості:
[атрибути] [специфікатори] тип <ім'я властивості>
{
[get код доступу]
[set код_доступу]
}
Код доступу являє собою блоки операторів, які виконуються при одержанні (get) або установці (set) властивості. Може бути відсутнім або частина get, або set, але не обидві одночасно. Якщо відсутня частина set, властивість доступно тільки для читання (read-only), якщо відсутня частина get, властивість доступно тільки для запису (write-only).

Білет №6

1) Перетворення типу бувають неявні(компілятором) і явні(прогамістом).
float f = 0;
double x = f; // таке присвоєння припустиме
f = x; // а таке – ні
f = (float)x; // явне приведення типу
Змінній x типу double можна присвоїти значення змінної менш "потужного" типу, наприклад, типу float, тому що компілятор C# виконує неявне приведення типу (explicit convert), а от розраховувати, що автоматично буде виконане зворотне перетворення – неможливо. Адже при присвоєнні значень більш потужного типу змінній менш потужного типу можливі втрати інформації. Відповідальність за виконання таких операцій в разі їх потреби програміст повинен взяти на себе, необхідно виконати

явне приведення типу (implicit convert). Така операція записується інструкцією:
(тип_до_якого_приводимо_вираз)вираз;
2) Наслідування – це процес завдяки якому один об’єкт може отримувати властивості другого. Завдяки наслідуванню підтримується концепція ієрархічної класифікації у вигляді якої організується більшість областей знань. Якщо б не використовувати ієрархічне подання ознак, для кожного об’єкта необхідно було б в явні формі визначити всі властиві йому характеристики. Але завдяки наслідуванню об’єкту необхідно до визначити лише ті якості, які роблять його унікальними всередині його класу, оскільки цей об’єкт наслідує загальні атрибути свого батьківського класу. class Class1{тіло класа} / class Class2 : Class1 {}

Білет №7

1) Операція простого присвоєння (=) використовується для заміни значення лівого операнда, значенням правого операнда. тип ім'я_змінної=вираз; Крім звичайної операції присвоєння (=) в мові C# виконуються ще 10 скорочених операцій присвоєння, які позначаються як = , де знак може бути замінений на знак однієї з операцій : + - * / % & ^ >> << . При цьому значення конструкції <змінна> = <вираз> ; обчислюється наступним чином: <змінна> = <змінна> <вираз> ; Наприклад, інструкція x += 1; еквівалента наступній інструкції: x = x + 1; . Оскільки виконання операції присвоєння генерує результат, рівний значенню своєї правої частини, допускається виконання ланцюжку присвоєнь. Так, наприклад, в наступному фрагменті коду ініціалізуються нулями відразу 3 змінні: int i, j, k; i = j = k = 0;

2) Ієрархія – це набір класів зв*язаних відношенням успадкування.
В основі механізму, що дозволяє створювати ієрархії класів, лежить принцип успадкування. Клас, що лежить в основі ієрархії, називається базовим. Класи, що спадковують властивості базового класу, називаються похідними. Похідні класи, у свою чергу, можуть бути базовими стосовно своїх спадкоємців, що в результаті приводить до ланцюжка успадкування. Процес утворення похідного класу на основі базового називається виводом класу. З одного базового класу можна вивести декілька похідних. Крім того, похідний клас може бути спадкоємцем декількох базових класів. Успадкування буває одиночним і множинним. При одиночному успадкуванні в кожного похідного класу є лише один базовий клас, а при множинному — декілька.

Білет №8

1) Бінарний оператор & попередньо визначений для цілочисельних типів і типу bool. Для цілочисельних типів оператор & виконує побітову операцію І для всіх своїх операндів. Якщо обидва відповідні біта операндів = 1 тоді двійковий розряд результату = 1, якщо хоча б 1 з бітів = 0 то результат = 0. 1 & 3 = 01 & 11 = 01 =1 Для операндів bool оператор & виконує логічну операцію І для всіх своїх операндів. Таким чином, значення false повертається тоді, коли хоча б 1 з операндів має значення false

2)Маскування методів базового класу викор у випадку наслідування класів. Якщо треба замаскувати метод у похідному класі, який отримується з базового, то потрібно використати ключове слово new.

```
A a = new A();
B b = new B();
a.Draw(); // виведе a
b.Draw(); // виведе b
  }
}
class A
{
   public void Draw() {
      Console.WriteLine("a");
   }
```

```
}
class B : A
{
   public new void Draw() {
      Console.WriteLine("b");
   }
}
```

Білет №9

1) Бінарні оператори | попередньо визначені для цілочисельних типів і типу bool. Для цілочисельних типів оператор | виконує побітову операцію АБО для всіх своїх операндів. Якщо обидва відповідні біта операндів = 0 тоді двійковий розряд результату = 0, якщо хоча б 1 з бітів = 1 то результат = 1. 1 | 2= 01 | 10 = 11 = 3 Для операндів bool оператор | виконує логічну операцію АБО для всіх своїх операндів. Таким чином, значення false повертається тільки тоді, коли обидва операнда мають значення false.

2) модифікатори доступу:
private – закритий доступ. У цьому випадку члени (дані або методи) класу є доступними в межах класу. Це означає, що доступ до private членів класу мають тільки змінні та методи цього класу;
public – відкритий доступ. У цьому випадку члени класу є доступними за межами класу. Це означає, що public-члени класу можна використовувати у всіх інших фрагментах коду – навіть тих, що визначені за межами класу;
protected – захищений доступ. Даний тип використовується у випадку спадковості при побудові ієрархії класів. У цьому випадку protected-члени класу є захищеними. Такі члени класу є відкритими в межах ієрархії класів, але закритими за межами ієрархії класів;
internal – визначає доступність члена класу у всіх файлах збірки і його недоступність за межами збірки. Це означає, що internal-члени класу є відомими тільки в самій програмі, але не за її межами.

Білет №10

1) Оператор порозрядного логічного або з виключенням, тобто ^ , - це бінарна операція, результат дії якої дорівнює 1, якщо число складаємих одиничних бітів непарне, якщо ж їх число парне, то результат дорівнює 0. Оператор ^ здатний працювати як із цілочисельними типами, так із bool.
5^6 = 101^110 = 011 = 3

2) Статичні члени класу існують в 1 екзеплярі і створюються відразу після запуску програми. Доступ надається наступним чином Назва_класу.Статичний член. При оголошенні їх не потрібно визначати. До статичних членів класу можна віднести:
Статичні конструктори - конструктори,що використовуються для ініціалізації статичних, незмінних і інших полів. Статичні поля/методи - це доступи до поля (або просто поле),виклик метода(або метод), які можна здійснити навіть тоді, коли даний тип не має жодного екземпляра.

Білет №11

1)Оператор порозрядного зсуву(<<,>>) це бінарна операція, яка застосовується тільки до цілих типів. Загальна форма операторів зсуву:
значення << кількість_біт
значення >> кількість_біт
де кількість_біт – кількість двійкових розрядів, на які зсувається вказане значення.
Операція зсуву вліво на n розрядів множить число на 2 в степені n. Зсув вправо на n розрядів означає ділення на 2 в степені n.

```
int a, b, c;
a = 5;
b = a >> 2; // b = 5/4 = 1
c = a << 1; // c = 5*2 = 10
```

2) перевантаження методу — один із засобів реалізації поліморфізму, що полягає в можливості створювати кілька реалізацій методу із тим же ім'ям проте з різною сигнатурою — з різною кількістю параметрів або з різним типом параметрів. Поширюєтся на методи одного і того ж класу, зокрема і на

конструктори класу. Перевантаження методів відноситься до класифікації статичного поліморфізму, в якому буде викликано найбільш підходящий метод, тобто, метод визначається на основі списку його аргументів. Перевантажуючи метод, створюється множина різних методів, які мають одне і те саме ім'я. Рішення, про те який метод буде викликано відбувається у період компіляції. Перевантаження методів не слід плутати з поліморфізмом, при якому потрібний метод обирається під час виконання, за допомогою віртуальних методів, замість статичних.

Білет №12

1) Наступні оператори виконують логічні операції з використанням логічних операндів:
Унарний ! - Логічне заперечення.
Бінарні & (логічне І), | (Логічне АБО), а також ^ (логічне АБО з виключенням) Ці оператори завжди обробляють обидва операнда.
Бінарні && (умовне логічне І) і || (Умовне логічне АБО) оператори. Ці оператори обчислюють другий операнд, тільки якщо це необхідно.
Для операндів цілочислових типів оператори &, | і ^ виконують побітові логічні операції.
Оператор ! виконує логічне заперечення операнда, повертаючи true, якщо операнд має значення false, і false, якщо операнд має значення true.
Оператор & обчислює логічне І для всіх своїх операндів. Результат операції x & y приймає значення true, якщо обидва оператори x і y мають значення true. В іншому випадку результат буде false.
Умовний оператор логічного І && також обчислює логічне І для своїх операндів, але він не обчислює другий операнд, якщо перший операнд має значення false.
Оператор ^ обчислює логічне АБО з виключенням для всіх своїх операндів, повертаючи true для x ^ y, якщо x має значення true і y має значення false або x має значення false і y має значення true. В іншому випадку результат буде false. Тобто для операндів bool оператор ^ повертає той же результат, що і оператор нерівності ! =.
Оператор | обчислює логічне АБО для всіх своїх операндів. Результат операції x | y приймає значення true, якщо хоча б один з операторів x або y має значення true. В іншому випадку результат буде false.
Умовний оператор логічного АБО || також обчислює логічне АБО для своїх операндів., але він не обчислює другий операнд, якщо результатом першого операнда є значення true.
2) Віртуальний метод — метод об'єкта, різний для базового класу і класу нащадка. Він може бути перевизначеним у класі нащадка. Перевизначення методу - це зміна його реалізації в класі спадкоємця. Перевизначивши метод, він працюватиме по-різному в базовому класі і класі спадкоємця, маючи при цьому одне і те ж ім'я та аргументи і тип повернення. Віртуальний метод оголошується за допомогою ключового слова virtual: public virtual void Metod(); . Оголосивши віртуальний метод, ми тепер можемо перевизначити його в класі спадкоємця. Для цього використовується ключове слово override: public override void Metod(arg){…}

Білет №13

1)Оператори відношення <, >, <=, >=, !=, == виконують операції,які порівнюють 2 операнди. В результаті ми отримуємо значення типу bool (true,false) 2 != 4 1 + 2 == 4

2) Модифікатор abstract вказує, що змінюваний елемент має відсутню або неповну реалізацію. Модифікатор abstract можна використовувати з класами, методами, властивостями, індексаторами і подіями. Модифікатор abstract у визначенні класу дозволяє вказати, що клас може бути тільки базовим класом для інших класів. Члени, помічені як абстрактні або включені в абстрактний клас, повинні бути реалізовані класами, похідними від абстрактного класу.

```
abstract class Shapes//оголошення абстрактного класу
{
   abstract public int Area();//оголошення абстрактного методу
}
```

Білет №15

1) Оператор if робить розгалуження програми залежно від результату перевірки деякої умови на істинність: if (умова) оператор1;
Параметр умова може бути будь-яким виразом, але найчастіше воно містить оператори порівняння. Якщо умова, що перевіряється, має істинне значення (true), виконується оператор1. А якщо ні, то (false) виконання програми переходить до оператора, що йде за умовним оператором.

2) Конструктор як і будь-який метод, може бути “перевантаженим”. Тобто, у класі може бути декілька конструкторів, які відрізняються між собою типами і (або) кількістю параметрів.

```
PersonsInfo pi1 = new PersonsInfo("Jack");
PersonsInfo pi2 = new PersonsInfo("Bobby","Robinson");
class PersonsInfo
{
string name,surname;
public PersonsInfo(string name)
{
   this.name=name;
   Console.WriteLine(name);
}
public PersonsInfo(string name, string surname)
{
   this.name=name;
   this.surname = surname;
   Console.WriteLine("{0} {1}",name,surname);
}
}
```

Білет №17

1) Оператор do while виконує оператор або блок операторів, поки певний вираз не прийме значення false. Тіло циклу має бути укладена в фігурні дужки {}, якщо воно не складається з одного оператора. В цьому випадку фігурні дужки необов'язкові. Оператор while виконує оператор або блок операторів, поки певний вираз не прийме значення false. Значення виразу while перевіряється перед кожним виконанням циклу, тому цикл while виконується нуль або кілька разів. Це відрізняє його від циклу do, який виконується від одного до декількох разів. Цикл while може бути завершений, якщо оператор break, goto, return або throw передає управління за межі циклу. Щоб передати управління в наступну ітерацію без виходу з циклу, використовуйте оператор continue. int n = 1 ;

```
while (n++ < 6 )
{
Console.WriteLine( "Current value of n is {0}" , n);
}
```

2) Якщо значення, яке явно перетворюється, лежить за межами діапазону значень типу призначення, то помилка не виникає, а результат перетворення залежатиме від конкретного типу. Для перевірки коректності перетворення використовують оператор checked:

```
int i = checked((int)l);
```

Якщо значення l виходить за межі значень типу int, то оператор checked згенерує виняткову ситуацію (переповнення). Типи за значенням допускають перетворення лише у числові типи, типи enum і тип char. Не можна безпосередньо перетворити тип bool до довільного іншого, і навпаки. Тип стрічки перетворюється в інші типи (і навпаки) за допомогою відповідних методів .NET. Оскільки кожен клас C# є нащадком класу object, то він успадковує (або має власну реалізацію) метод ToString():

```
int i = 1; string s = i.ToString();
```

Для переводу стрічкового значення у числове або значення типу bool використовують метод Parse:

```
string s = "1"; int i = Int32.Parse(s);
```

Білет №18

1) Оператор foreach застосовується для перебору елементів у колекції/масиві. Зручність цього виду циклу полягає в тому, що нам не потрібно визначати кількість елементів у групі й виконувати їхній перебір по індексу.

Синтаксис оператора:

foreach (тип ім'я in вираз) {тіло циклу} Впроваджені оператори продовжують виконуватися для кожного елемента масиву або збірки. Після завершення ітерації всіх елементів збірки управління переходить до наступного оператору після блоку foreach.

2)Усі члени базового класу автоматично стають членами похідного. Керуючись оголошенням похідного класу, компілятор ніби збирає його з різних частин — спочатку він бере усі властивості базового класу, а потім додає до них нові функціональні можливості похідного. Для цього використовується наступна синтаксична конструкція. class
ім'я_похідного_класу:специфікатор_доступу ім'я_базового_класу { // тіло класу };
Хоча всі члени базового класу автоматично стають членами похідного класу, однак доступ до цих членів визначається видом успадкування. У залежності від специфікатора доступу, зазначеного при оголошенні похідного класу, розрізняють відкрите, закрите і захищене успадкування. За замовчуванням використовується закрите успадкування (специфікатор private).

Білет №19

1) Оператор is перевіряє сумісність об'єкта з даним типом, а в якості результату видає значення типу bool. Оператор is ніколи не генерує виняток, він працює з усіма типами.
Оператор as працює зі посилальними типами, він перетворює типи, якщо це можливо і повертає null, якщо неможливо. Команда A a = o as A; буде виконана успішно, але в посилання a буде записаний null, і будь-яке звернення до a як до об'єкта, наприклад, a.ToString (); викличе генерацію винятка. Перевірку того, чи посилання не отримало нульового значення, повинен виконати програміст.

Білет №20

1) break - оператор,що припиняє роботу операторів циклу(while,for,foreach),switch,передаючи керування наступній стрічці коду.(?Використовується для "виходу"). Наведемо приклад:

```
while(1)
{
//код
if(<вираз1>)
{
break;
}
//код
}
<оп1>
//код
```

continue - оператор,що передає керування на наступну ітерацію цилку.(Використовується аби перейти на наступний "крок" цилку). Наведемо приклад:

```
int i=1;
while(i<10)
{
//код1
if(<вираз1>)
{
continue;
}
//код2
}
```

return - оператор , що завершує виконування метода, у якому він присутній .Проте також може повертати значення у методах не пустого типу,тобто у методах ,що повертають якесь значення (int,double,string,byte...),властивостях,індексаторах.
Приклад:

```
int i(int a,int b)
{
return a+b;
}
```

goto - оператор, що передає керування <оп> , на індетифікатор, що позначається як <мітка>, ігноруючи весь код,що є між ними і після виконання <оп1>, не повертається, а виконує дії , що йдуть уже після мітки,

і варто зауважити,що ця передача керування працює лише у межах методу. Синтаксис - goto<мітка>; <мітка>:<оп1>;

```
int i()
{
int a=0;
goto A;
a++;
Console.WriteLine(a);
A:
return a;
}
```

2)Якщо властивості — це "розумні" поля, то індексатори – це "розумні" масиви, тому що вони дозволяють індексувати об'єкти методами-аксессорами get і set. За допомогою індексатора легко проіндексувати об'єкт для установки або одержання значень. Також можна сказати,що індексатор це вид властивості,що дозволяє працювати з екземпляром як з масивом, ключове слово якого є this (- воно "каже" ,що індексатор належить класу, це не ім'я ?). Наведемо приклад: public int this[int index] { get{} set{} }

Білет №22

2) Одновимірний масив по-іншому ще називається вектором, і для доступу до його елементів використовується тільки один індекс. У C# оголошення масиву має таку структуру:
тип[] ім’я_масиву = new тип[розмір масиву];
Всім елементам при створенні масиву привласнюються значення за замовчуванням: нулі для значимих типів й null - для посилальних. Кількість елементів у масиві (розмірність) не є частиною його типу, ця кількість задається при виділенні пам'яті й не може бути змінене згодом. Розмірність може задаватися не тільки константою, але й виразом. Результат обчислення цього виразу повинен бути не від’ємним, а його тип повинен мати неявне перетворення до int, uint, long або ulong.
Приклад:
int[] array = new int[5];
Якщо відбувається ініціалізація, оператор new можна опускати. Ініціалізація масиву дозволяє записати значення в комірки масиву в момент його оголошення в програмі.
int [] сезони = {1,2,3,4};
Доступ до елементів здійснюється за індексом.
Індексація починається з нуля - перший елемент масиву має індекс 0, а останній N-1, де N - розмір масиву. Всі масиви в C# мають загальний базовий клас Array, визначений у просторі імен System.

Білет №23

2) Багатовимірний масив – це масив, що має два і більше вимірів. Для доступу до елементу багатовимірного масиву використовується комбінація з двох і більше індексів. Загальна форма оголошення багатовимірного масиву наступна:
тип[,...,] ім’я_масиву = new тип[розмір1, розмір2, ..., розмірN];
Всім елементам при створенні масиву привласнюються значення за замовчуванням: нулі для значимих типів й null - для посилальних. Кількість елементів у масиві (розмірність) не є частиною його типу, ця кількість задається при виділенні пам'яті й не може бути змінене згодом. Розмірність може задаватися не тільки константою, але й виразом. Результат обчислення цього виразу повинен бути не від’ємним, а його тип повинен мати неявне перетворення до int, uint, long або ulong. Якщо відбувається ініціалізація, оператор new можна опускати. Ініціалізація масиву дозволяє записати значення в комірки масиву в момент його оголошення в програмі.
int[,] numbers1 = new int[2, 2]; // оголошення двовимірного масиву
int[,,] numbers2 = new int[2, 2 ,3]; // оголошення трьохвимірного масиву
int[,] numbers3 = new int[3, 2] { {6, 0}, {5, 7}, {8, 9} }; // ініціалізація двовимірного масиву

Білет №26

2) Для реалізації інтерфейсу клас повинен забезпечити тіла (способи реалізації) методів, описаних в інтерфейсі. Кожен клас може визначити власну реалізацію. Таким чином, два класу можуть реалізувати один і той же інтерфейс різними способами, але всі класи підтримують однаковий набір

методів. Отже, код, "обізнаний" про наявність інтерфейсу, може використовувати об'єкти будь-якого класу, оскільки інтерфейс для всіх об'єктів однаковий. Надаючи програмістам можливість застосування такого засобу програмування, як інтерфейс, C# дозволяє повною мірою використовувати аспект поліморфізму, який виражається як "один інтерфейс — багато методів". Інтерфейси оголошуються за допомогою ключового слова interface. Ось як виглядає спрощена форма оголошення інтерфейсу:

```
interface ім'я{
тип_повернення імя_методу1(список_параметрів);
тип_повернення імя_методу2(список_параметрів);
.................
тип__повернення імя_методуN(слисок_параметрів);
}
```

Ім'я користувача задається елементом ім'я. Методи оголошуються з використанням лише типу повертається значення і сигнатури. Всі ці методи, по суті, — абстрактні. Як згадувалося вище, для методів в інтерфейсі не передбачені способи реалізації. Отже, кожен клас, який включає інтерфейс, повинен реалізувати всі його методи. В інтерфейсі методи неявно є відкритими (public-методами), при цьому не дозволяється явно вказувати специфікатор доступу.

Білет №27

2) виклик методів інтерфейсу
(хз чи це норм)
Виклик методів інтерфесу є неймовірно важливий,адже на цьому базується мета інтерфейсу.
Приклад:

```
class Try {
static void Main() {
Car car = new Car();
IMove i = car;
i.Move();
Console.WriteLine(i.velocity());
}
}
class Car : IMove
{
int speed=40;
public void Move()
{
Console.WriteLine("Drive");
}
public int velocity()
{
return speed;
}
}
interface IMove
{
void Move();
}
```

((?! Хз чи це норм)Тут ми бачимо , що виклик методу Інтерфейсу Move(); виконається, так цей метод по суті є абстрактий , і при виклику методу Move(); у інтерфейсі , ? насправді виконується Move() з класу Car , так як інтерфейс присвоюється посиланню на об’єкт Car.)
Хз2.
Якщо клас реалізує два інтерфейси, що містять член з однаковою сигнатурою, при реалізації такого члена в класі обидва інтерфейсу будуть використовувати цей член в якості власної реалізації. У наступному прикладі всі виклики Paint призводять до виклику одного і того ж методу.

```
class Test
{
static void Main()
{
SampleClass sc = new SampleClass();
```

```
IControl ctrl = (IControl)sc;
ISurface srfc = (ISurface)sc;

// The following lines all call the same method.
sc.Paint();
ctrl.Paint();
srfc.Paint();
}
}


interface IControl
{
void Paint();
}
interface ISurface
{
void Paint();
}
class SampleClass : IControl, ISurface
{
// Both ISurface.Paint and IControl.Paint call this method.
public void Paint()
{
Console.WriteLine("Paint method in SampleClass");
}
}

// Output:
// Paint method in SampleClass
// Paint method in SampleClass
// Paint method in SampleClass
```

Член класу IControl.Paint доступний тільки через інтерфейс IControl, а ISurface.Paint - тільки через інтерфейс ISurface. Обидві реалізації методу будуть розділені, і жодна з них не буде доступна в класі безпосередньо.

```
// Call the Paint methods from Main.

SampleClass obj = new SampleClass();
//obj.Paint(); // Compiler error.

IControl c = (IControl)obj;
c.Paint(); // Calls IControl.Paint on SampleClass.

ISurface s = (ISurface)obj;
s.Paint(); // Calls ISurface.Paint on SampleClass.

// Output:
// IControl.Paint
// ISurface.Paint
```

Білет №29

2) Виняток - об'єкти класу Exception або його нащадків, у яких передається інформація про помилки. Для обробки виняткових ситуацій використовується конструкція try/catch, що має наступний синтаксис:

try {програмний код, при виконанні якого можливі помилки}
catch (тип змінна) {програмний код для обробки помилки зазначеного типу}

Можна перехоплювати не один, а кілька типів винятків, якщо використовувати кілька блоків catch. Перехопивши виняток, можна отримати більш детальну інформацію про помилку. Властивість Message містить короткий опис помилки; властивість TargetSite представляє об'єкт, в якому виникла помилка (використовуючи цей об'єкт, можна отримати ім'я методу, ім'я класу та назва збірки, в якій

виникла помилка); властивість InnerException вертає наступний об'єкт в ланцюжку винятків.
У тому випадку, коли в програмному коді обробки помилки немає необхідності в отриманні детальної інформації про помилку, застосовується спрощена конструкція try/catch:
try {програмний код, при виконанні якого можливі помилки}
catch {програмний код для обробки помилки зазначеного типу}
Також можна використовувати конструкцію try/catch/finally, що має наступний синтаксис:
try {програмний код, при виконанні якого можливі помилки}
catch (тип змінна) {програмний код для обробки помилки зазначеного типу}
finally {програмний код, що виконується незалежно від помилки}

Білет №30

2) Делегат - клас, в якому зберігається метод. В визначення делегата входить прототип методів, які можуть бути представлені цим делегатом. При оголошенні змінної типу делегата вказується метод, тип якого сумісний з типом делегата. Після цього делегат можна використовувати для виклику зазначеного метода. Зазвичай делегати використовуються для реалізації зворотних викликів. Оголошення делегата може розташовуватися всередині або поза будь-яким класом і схоже на оголошення методу, але з додаванням до початку ключового слова delegate. Для оголошення делегата використовується наступний синтаксис:
delegate тип делегат(аргументи);
Можливість оголошення делегата поза будь-яким класом пов'язана з тим, що на основі рядка оголошення делегата компілятор фактично генерує оголошення класу, похідного від MulticastDelegate.
Для створення делегата необхідно визначити змінну типу делегата і за допомогою ключового слова new викликати конструктор делегата, передавши йому в якості параметра ім'я методу, який зберігатиметься в делегаті. делегат змінна=new делегат(об*кт.метод);
Для виклику делегата вказується ім'я змінної типу делегата, а потім у круглих дужках перераховуються через кому параметри, які слід передати методу, що викликається через об'єкт делегата. В основі механізму обробки **подій** лежать делегати, тому, перш ніж оголосити подію, необхідно оголосити делегат. Оголошення події схоже на оголошення змінної типу делегата, але з додаванням до початку ключового слова event. Для оголошення події використовується наступний синтаксис:
event делегат подія;
Щоб згенерувати подію, вказується ім'я події, а потім у круглих дужках перераховуються через кому параметри події. Перш ніж затронути подію, рекомендується перевірити чи не дорівнює null змінна, що описує подію. Для підписки на подію для будь-якого об'єкта використовується оператор +=, після якого за допомогою ключового слова new викликається конструктор делегата, як параметр якого вказується назва методу, що представляє з себе обробник подій.
об*єкт.подія+=new делегат(метод);